



Name: Lobna Ahmed Abd Elhamid

ID: 2300360

Sec: 2

Course: [CSE335s] Operating Systems

Memory Segmentation Report

[Project Files on Google Drive](#)

1. Project Overview

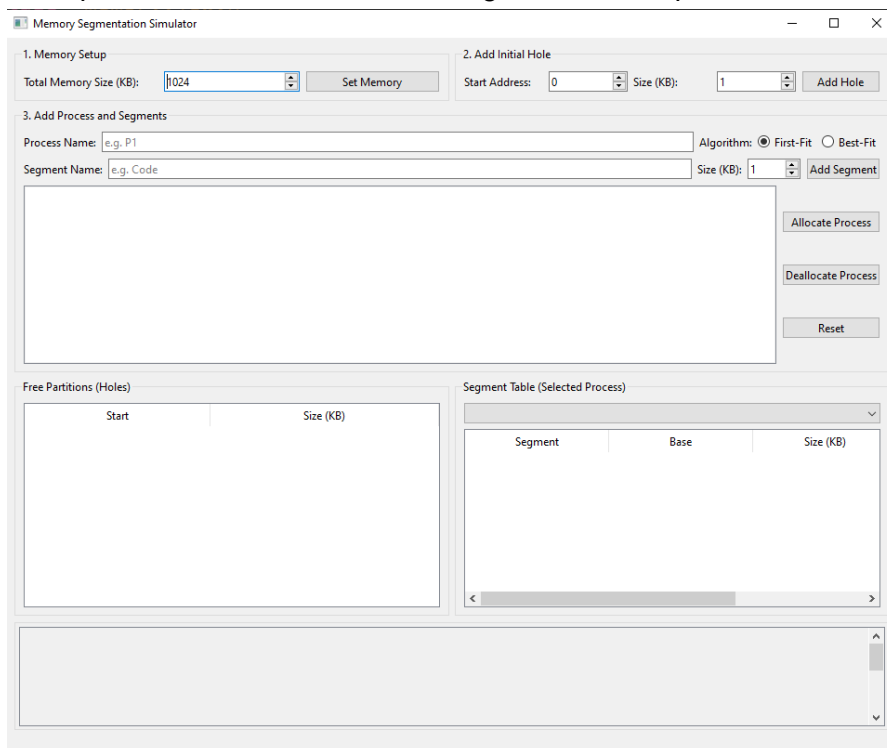
This project implements a Memory Segmentation Simulator using C++ and Qt for the graphical user interface. The simulator manages memory allocation and deallocation using the segmentation memory management scheme, which supports the user view of a program as a collection of named segments.

The simulator supports two allocation algorithms:

- First-Fit: allocates the first hole large enough to fit the segment.
- Best-Fit: allocates the smallest hole that is still large enough, minimizing wasted space.

Key features implemented:

- Dynamic allocation and deallocation of process segments.
- Automatic merging of adjacent free holes after deallocation.
- Live memory layout visualization showing allocated segments and free holes.
- Segment table display per process showing base address and size of each segment.
- Input validation and error handling for all user inputs.



2. Data Structures

The project uses four core structs defined in mainwindow.h:

2.1 Segment Struct

Represents a single named segment belonging to a process:

```
struct Segment {
    QString name;           // segment name: Code, Data, Stack
    int size;               // requested size in KB
    int baseAddress;        // starting address in memory (-1 = unallocated)
    bool allocated;         // true if currently in memory
};
```

2.2 Process Struct

Represents a process containing one or more segments:

```
struct Process {
    int id;                 // unique process identifier
    QString name;           // process name e.g. P1
    QVector<Segment> segments; // all segments of this process
    bool fullyAllocated;    // true only if ALL segments fit
};
```

2.3 Hole Struct

Represents a free memory partition available for allocation:

```
struct Hole {
    int startAddress; // starting address of the free region
    int size;         // size of the free region in KB
};
```

2.4 MemoryBlock Struct (memoryview.h)

Used by the visual memory bar to represent each block (allocated or hole):

```
struct MemoryBlock {
    int startAddress;
    int size;
    QString label; // shown inside the block
    bool isHole;   // gray if hole, colored if allocated
    int pid;       // process id for color assignment
};
```

3. Core Functions

3.1 firstFit()

Scans the holes vector from the beginning and returns the index of the first hole large enough to fit the requested segment size. Returns -1 if no hole fits.

```
int MainWindow::firstFit(int segmentSize) {
    for (int i = 0; i < holes.size(); i++) {
        if (holes[i].size >= segmentSize)
            return i;
    }
}
```

```

    }
    return -1;
}

```

3.2 bestFit()

Scans all holes and returns the index of the smallest hole that is still large enough. This minimizes wasted space after allocation. Returns -1 if no hole fits.

```

int MainWindow::bestFit(int segmentSize) {
    int bestIdx = -1, bestSize = INT_MAX;
    for (int i = 0; i < holes.size(); i++) {
        if (holes[i].size >= segmentSize && holes[i].size < bestSize) {
            bestSize = holes[i].size; bestIdx = i;
        }
    }
    return bestIdx;
}

```

3.3 allocateProcess()

Implements a two-phase allocation strategy. Phase 1 simulates the allocation on a temporary copy of the holes vector. If all segments of the process fit, Phase 2 commits the changes to the real holes vector. If any segment does not fit, the entire process is rejected and holes remain unchanged.

```

void MainWindow::allocateProcess(Process& p) {
    QVector<Hole> tempHoles = holes;           // Phase 1: simulate
    QVector<QPair<int,int>> placements;
    bool allFit = true;
    for (int i = 0; i < p.segments.size(); i++) {
        int chosenIdx = rb_firstFit ? firstFit(...) : bestFit(...);
        if (chosenIdx == -1) { allFit = false; break; }
        placements.append({chosenIdx, tempHoles[chosenIdx].startAddress});
        tempHoles[chosenIdx].startAddress += segSize;
        tempHoles[chosenIdx].size -= segSize;
        if (tempHoles[chosenIdx].size == 0) tempHoles.remove(chosenIdx);
    }
    if (!allFit) { reject process; return; }
    holes = tempHoles;                         // Phase 2: commit
    // assign base addresses to segments
}

```

3.4 deallocateProcess()

Finds the process by ID, converts each of its allocated segments back into holes, removes the process from the processes vector, and calls mergeHoles() to combine any adjacent free regions.

```

void MainWindow::deallocateProcess(int processId) {
    // find process → convert segments to holes
    for each segment: holes.append({seg.baseAddress, seg.size});
    processes.remove(processId);
    mergeHoles(); }

```

3.5 mergeHoles()

Sorts holes by start address, then scans for adjacent holes. Two holes are adjacent when the end of one equals the start of the next. Adjacent holes are merged into a single larger hole to prevent fragmentation.

```
void MainWindow::mergeHoles() {
    std::sort(holes by startAddress);
    int i = 0;
    while (i < holes.size() - 1) {
        if (holes[i].start + holes[i].size == holes[i+1].start) {
            holes[i].size += holes[i+1].size;
            holes.remove(i+1);
        } else i++;
    }
}
```

4. Memory Layout Visualization

The MemoryView class (memoryview.h / memoryview.cpp) is a custom Qt widget that draws the memory bar using QPainter. It receives a sorted list of MemoryBlock objects and renders each as a proportionally scaled rectangle.

- Allocated segments are colored uniquely per process using HSV color space.
- Free holes are drawn in gray with their size labeled inside.
- Address labels are drawn below the bar using a QMap to collect and deduplicate addresses, with a minimum pixel spacing of 35px to prevent overlap.
- A tick line is drawn at each address boundary for clarity.

5. Test Cases

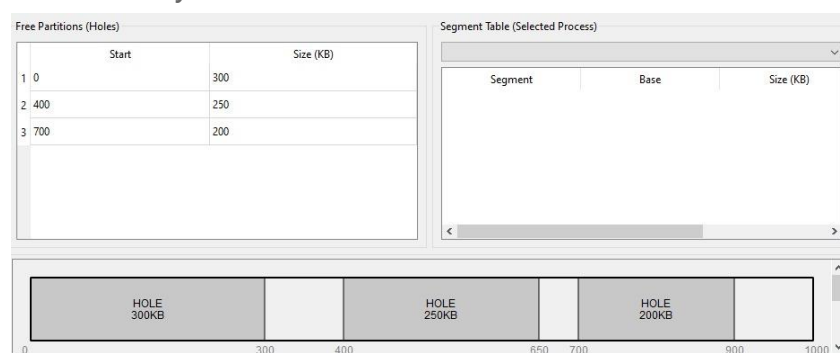
The following scenario is run twice — once using First-Fit and once using Best-Fit — to compare allocation behavior.

Initial Memory Setup

Hole	Start Address (KB)	Size (KB)
H1	0	300
H2	400	250

H3	700	200
----	-----	-----

Total Memory Size: 1000 KB



6. First-Fit Allocation

Operation 1: Allocate P1 (Code=100, Data=120, Stack=90)

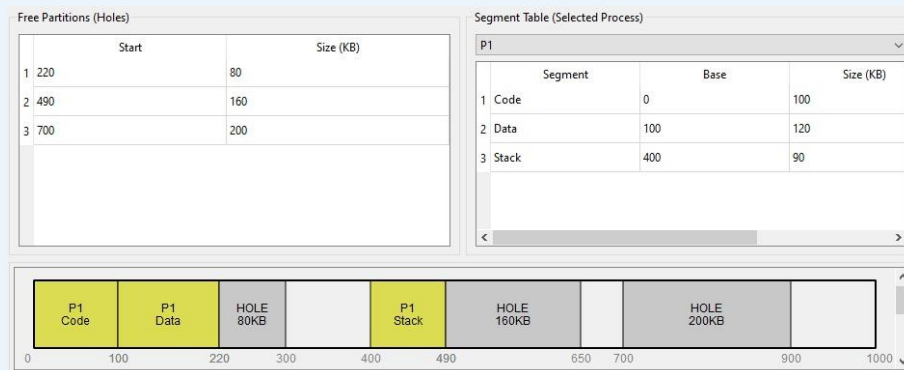
Holes before allocation: H1(0, 300), H2(400, 250), H3(700, 200)

First-Fit decisions:

- Code (100 KB): First hole with size ≥ 100 is H1(300) -> placed at 0. H1 becomes (100, 200).
- Data (120 KB): First hole with size ≥ 120 is H1(200) -> placed at 100. H1 becomes (220, 80).
- Stack (90 KB): First hole with size ≥ 90 is H2(250) -> placed at 400. H2 becomes (490, 160).

Segment	Size (KB)	Hole Used	Base Address
Code	100	H1	0
Data	120	H1	100
Stack	90	H2	400

Memory layout after P1 allocation



Operation 2: Allocate P2 (Code=200, Data=40)

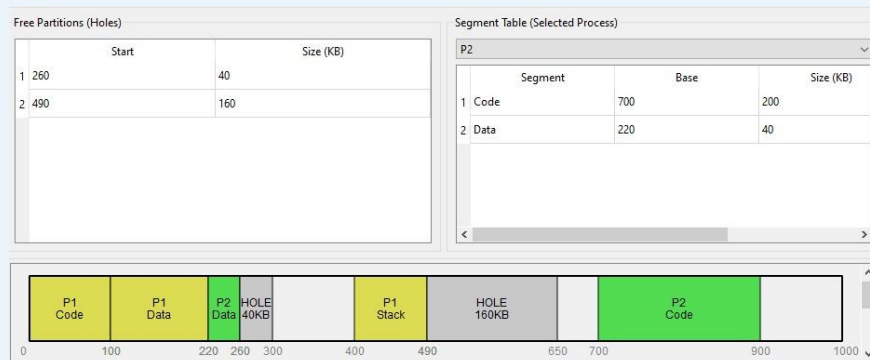
Holes before allocation: H1_remaining(220, 80), H2_remaining(490, 160), H3(700, 200)

First-Fit decisions:

- Code (200 KB): H1(80) too small, H2(160) too small, H3(200) fits -> placed at 700. H3 becomes (900, 0) -> removed.
- Data (40 KB): H1(80) ≥ 40 -> placed at 220. H1 becomes (260, 40).

Segment	Size (KB)	Hole Used	Base Address
Code	200	H3	700
Data	40	H1	220

Memory layout after P2 allocation

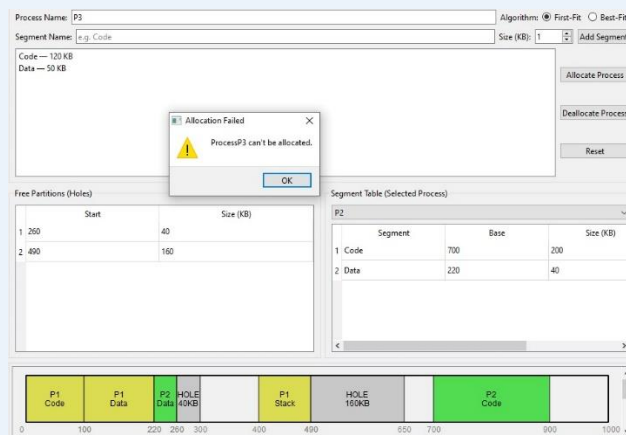


Operation 3: Allocate P3 (Code=120, Data=50)

First-Fit decisions:

- Code (120 KB): H1(40) too small, H2(160) ≥ 120 -> placed at 490. H2 becomes (610, 40).
- Data (50 KB): H1(40) too small, H2(40) too small -> no hole fits.
- Result: P3 cannot be allocated - rejected entirely. Memory unchanged.

Rejection message for P3



Operation 4: Deallocate P1

Segments freed: Code(0,100), Data(100,120), Stack(400,90)

These become new holes. After mergeHoles():

- Code(0,100) + Data(100,120) -> adjacent -> merged into (0, 220).
- (0,220) + H1_remaining(220,40) -> adjacent -> merged into (0, 260).
- Stack(400,90) + H2_remaining(490,160) -> adjacent -> merged into (400, 250).

Memory layout after P1 deallocation

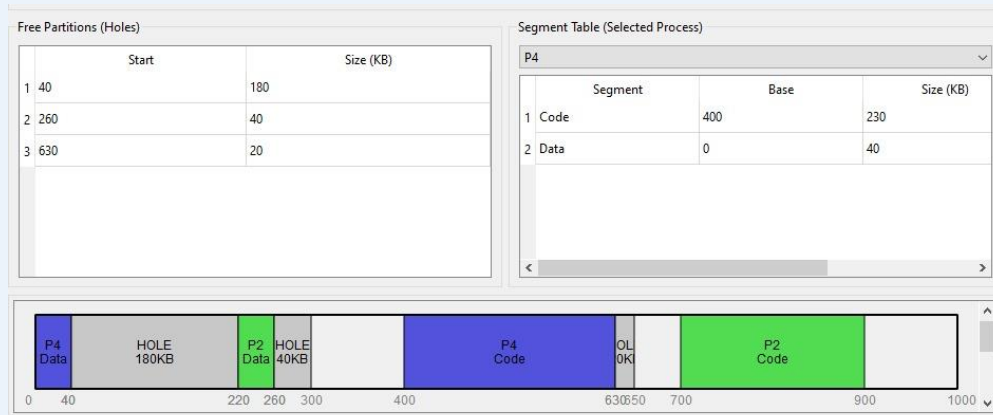


Operation 5: Allocate P4 (Code=230, Data=40)

First-Fit decisions:

- Code (230 KB): (0,260) $\geq$ 230 -> placed at 0. Hole becomes (230, 30).
- Data (40 KB): (230,30) too small, (260,40) $\geq$ 40 -> placed at 260. Hole removed.

Memory layout after P4 allocation



7. Best-Fit Allocation

Same operations repeated using Best-Fit algorithm. Best-Fit selects the smallest hole that is still large enough for the segment.

Operation 1: Allocate P1 (Code=100, Data=120, Stack=90)

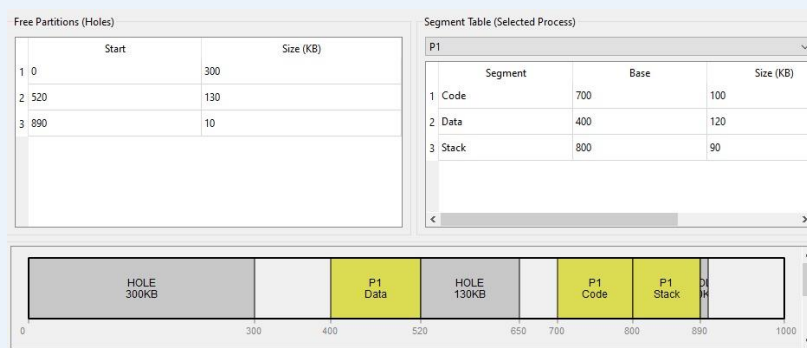
Holes: H1(0,300), H2(400,250), H3(700,200)

Best-Fit decisions:

- Code (100 KB): H1(300), H2(250), H3(200) all fit → smallest is H3(200) → placed at 700. H3 becomes (800,100).
- Data (120 KB): H1(300), H2(250), H3(100 too small) → smallest fitting is H2 was removed, now H2(250) is smallest → placed at 400. H2 becomes (520,130).
- Stack (90 KB): H1(300), H2(130), H3(100) → smallest fitting is H3(100) → placed at 800. H3 becomes (890,10).

Segment	Size (KB)	Hole Used	Base Address
Code	100	H3(700,200)	700
Data	120	H2(400,250)	400
Stack	90	H3(800,100)	800

Memory layout after P1 allocation



Operation 2: Allocate P2 (Code=200, Data=40)

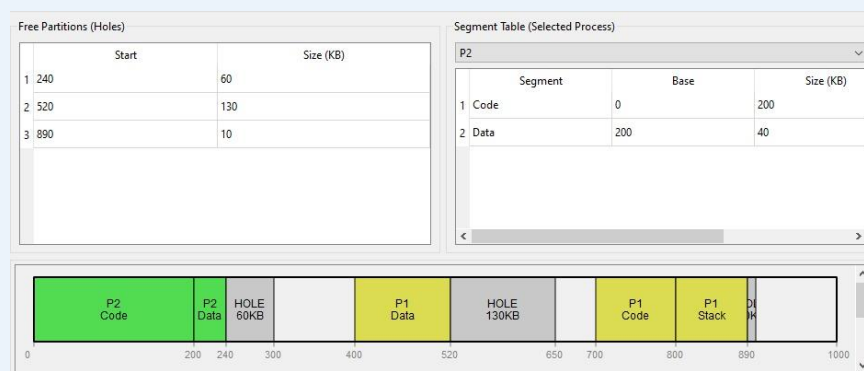
Holes: H1(0,300), H2(520,130), H3(890,10)

Best-Fit decisions:

- Code (200 KB): H1(300) fits, H2(130) too small → only H1 fits → placed at 0. H1 becomes (200,100).
- Data (40 KB): H1(200,100) fits, H2(520,130) fits → smallest fitting is H1(100) → placed at 200. H1 becomes (240,60).

Segment	Size (KB)	Hole Used	Base Address
Code	200	H1(0,300)	0
Data	40	H1(200,100)	200

Memory layout after P2 allocation



Operation 3: Allocate P3 (Code=120, Data=50)

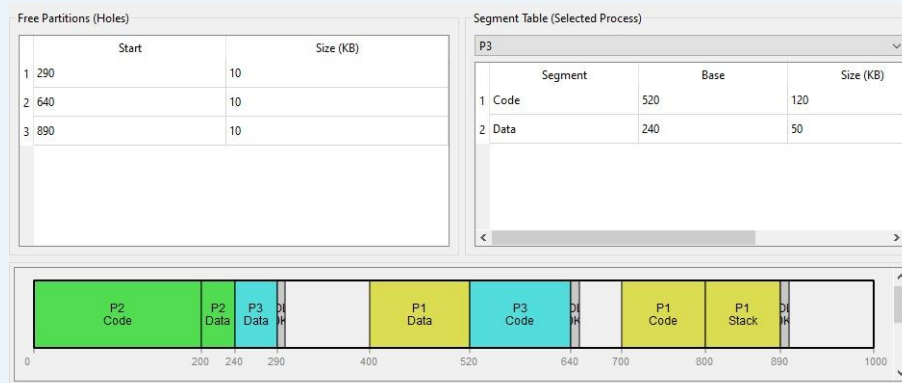
Holes: H1(240,60), H2(520,130), H3(890,10)

Best-Fit decisions:

- Code (120 KB): H1(60) too small, H2(130) ≥ 120 → placed at 520. H2 becomes (640,10).
- Data (50 KB): H1(60) ≥ 50 → placed at 240. H1 becomes (290,10).

Segment	Size (KB)	Hole Used	Base Address
Code	120	H2(520,130)	520
Data	50	H1(240,60)	240

Memory layout after P3 allocation



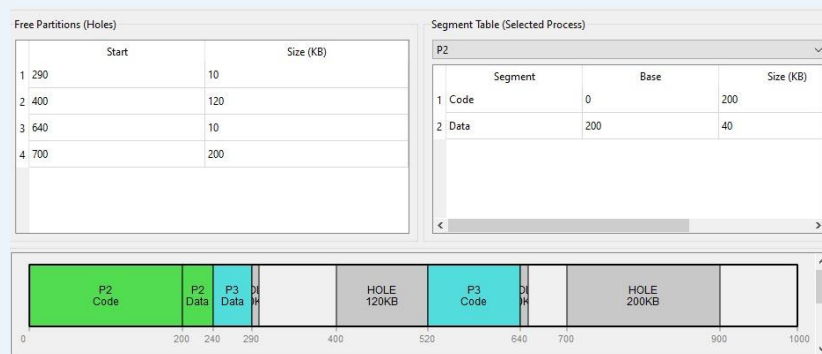
Operation 4: Deallocate P1

Segments freed: Code(700,100), Data(400,120), Stack(800,90)

After mergeHoles():

- Code(700,100) + Stack(800,90) -> adjacent -> merged into (700,190).
- Data(400,120) -> standalone hole at (400,120).

Memory layout after P1 deallocation



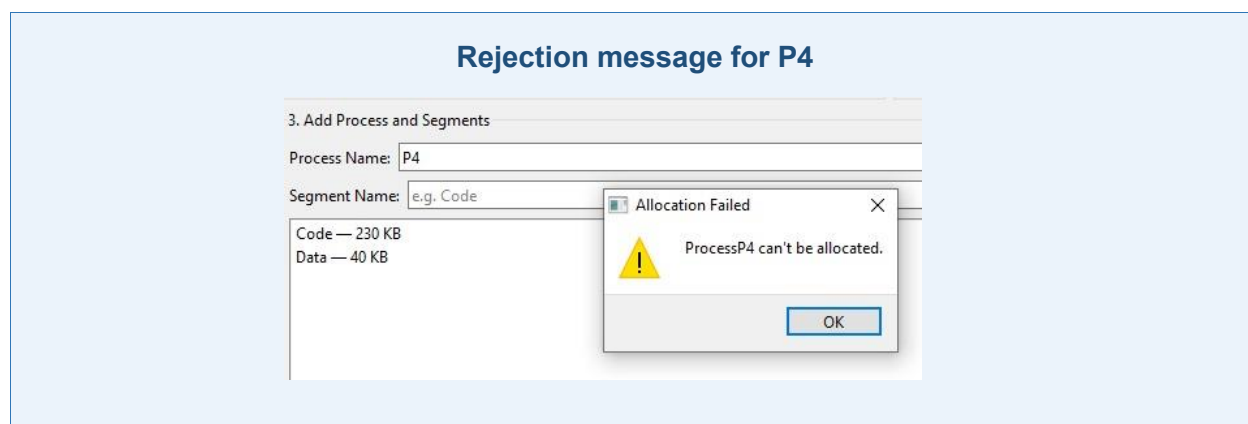
Operation 5: Allocate P4 (Code=230, Data=40)

Available holes after deallocation: (290,10), (400,120), (640,10), (700,190), (890,10)

Best-Fit decisions:

- Code (230 KB): Only (700,190) could be close - $190 < 230$, too small. No hole fits.

- Result: P4 cannot be allocated using Best-Fit - rejected. Memory unchanged.



8. Algorithm Comparison

Operation	First-Fit Result	Best-Fit Result
Allocate P1	Success	Success
Allocate P2	Success	Success
Allocate P3	Failed	Success
Deallocate P1	Success	Success
Allocate P4	Success	Failed

This comparison demonstrates that neither algorithm is universally superior. First-Fit is faster but may leave small unusable holes early in memory. Best-Fit minimizes wasted space per allocation but can create many tiny holes that collectively cause more fragmentation over time.